

Bottom-Up Parsing

- ◆ A **bottom-up parser** creates the parse tree of the given input starting from leaves towards the root.
- ◆ A bottom-up parser tries to find the right-most derivation of the given input in the reverse order.
$$S \Rightarrow \dots \Rightarrow \omega \quad (\text{the right-most derivation of } \omega)$$
$$\leftarrow (\text{the bottom-up parser finds the right-most derivation in the reverse order})$$
- ◆ Bottom-up parsing is also known as **shift-reduce parsing** because its two main actions are shift and reduce.
 - At each **shift** action, the current symbol in the input string is pushed to a stack.
 - At each **reduction** step, the symbols at the top of the stack (this symbol sequence is the right side of a production) will be replaced by the non-terminal at the left side of that production.
 - There are also two more actions: accept and error.

Shift-Reduce Parsing

A shift-reduce parser tries to reduce the given input string into the starting symbol.

a string $\rightarrow$ the starting symbol
reduced to

- ◆ At each reduction step, a substring of the input matching to the right side of a production rule is replaced by the non-terminal at the left side of that production rule.
- ◆ If the substring is chosen correctly, the right most derivation of that string is created in the reverse order.

Rightmost Derivation:

$$S \xRightarrow[\text{rm}]{*} \omega$$

Shift-Reduce Parser finds:

$$\omega \xleftarrow{\text{rm}} \dots \xleftarrow{\text{rm}} S$$

reduction 

Shift-Reduce Parsing – Example

$S \rightarrow aABb$

$A \rightarrow aA \mid a$

$B \rightarrow bB \mid b$

input string: **aaabb**

aaAbb

aAbb

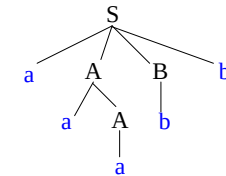
aABb

S

⇓ reduction

$S \Rightarrow aABb \Rightarrow aAbb \Rightarrow aaAbb \Rightarrow aaabb$

rm rm rm rm



Right Sentential Forms

How do we know which substring to be replaced at each reduction step?

Handle

- ◆ Informally, a **handle** (句柄) of a string is a substring that matches the right side of a production rule.
 - But not every substring matches the right side of a production rule is handle
- ◆ A handle of a right sentential form $\gamma (\equiv \alpha\beta\omega)$ is a production rule $A \rightarrow \beta$ and a position of γ where the string β may be found and replaced by A to produce the previous right-sentential form in a rightmost derivation of γ .

$$S \xRightarrow[\text{rm}]{*} \alpha A \omega \Rightarrow_{\text{rm}} \alpha \beta \omega$$

- ◆ If the grammar is unambiguous, then every right-sentential form of the grammar has exactly one handle.
- ◆ We will see that ω is a string of terminals.

Handle Pruning

- ◆ A right-most derivation in reverse can be obtained by **handle-pruning**.

$$S = \gamma_0 \Rightarrow \underset{\text{rm}}{\gamma_1} \Rightarrow \underset{\text{rm}}{\gamma_2} \Rightarrow \dots \Rightarrow \underset{\text{rm}}{\gamma_{n-1}} \Rightarrow \underset{\text{rm}}{\gamma_n} = \omega$$

input string

- ◆ Start from γ_n , find a handle $A_n \rightarrow \beta_n$ in γ_n , and replace β_n in by A_n to get γ_{n-1} .
- ◆ Then find a handle $A_{n-1} \rightarrow \beta_{n-1}$ in γ_{n-1} , and replace β_{n-1} in by A_{n-1} to get γ_{n-2} .
- ◆ Repeat this, until we reach S .

A Shift-Reduce Parser

$$E \rightarrow E+T \mid T$$

$$T \rightarrow T*F \mid F$$

$$F \rightarrow (E) \mid id$$

Right-Most Derivation of $id+id*id$

$$E \Rightarrow E+T \Rightarrow E+T*F \Rightarrow E+T*id \Rightarrow E+F*id$$

$$\Rightarrow E+id*id \Rightarrow T+id*id \Rightarrow F+id*id \Rightarrow id+id*id$$

Right-Most Sentential Form

id+id*id

E+id*id

T+id*id

E+id*id

E+F*id

E+T*id

E+T*F

E+T

E

Reducing Production

$F \rightarrow id$

$T \rightarrow F$

$E \rightarrow T$

$F \rightarrow id$

$T \rightarrow F$

$F \rightarrow id$

$T \rightarrow T*F$

$E \rightarrow E+T$

Handles are red and underlined in the right-sentential forms.

A Stack Implementation of A Shift-Reduce Parser



There are four possible actions of a shift-parser action:

1. **Shift** : The next input symbol is shifted onto the top of the stack.
2. **Reduce**: Replace the handle on the top of the stack by the non-terminal.
3. **Accept**: Successful completion of parsing.
4. **Error**: Parser discovers a syntax error, and calls an error recovery routine.

- ◆ Initial stack just contains only the end-marker \$.
- ◆ The end of the input string is marked by the end-marker \$.

Initial stack 

A Stack Implementation of A Shift-Reduce Parser



Stack

\$
\$**id**
\$**F**
\$**T**
\$E
\$E+
\$E+**id**
\$E+**F**
\$E+T
\$E+T*
\$E+T***id**
\$E+**T***F
\$**E**+T
\$E

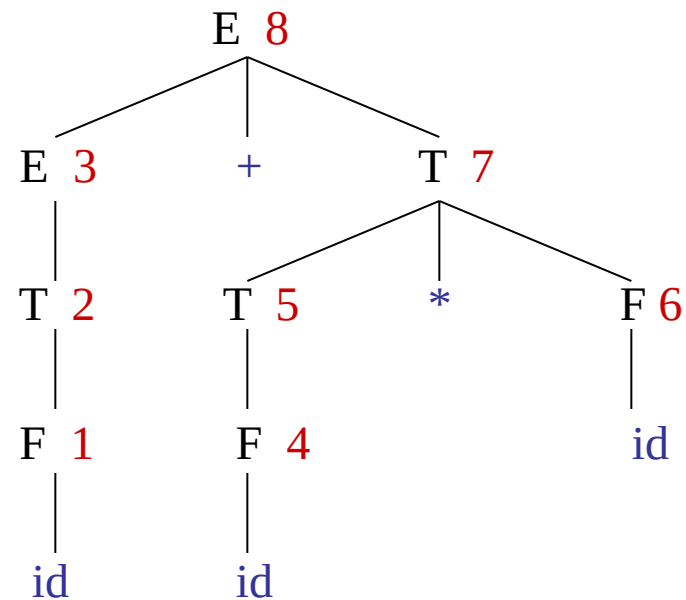
Input

id+id*id\$
+id*id\$
+id*id\$
+id*id\$
+id*id\$
id*id\$
*id\$
*id\$
*id\$
id\$
\$
\$
\$
\$

Action

shift
reduce by $F \rightarrow id$
reduce by $T \rightarrow F$
reduce by $E \rightarrow T$
shift
shift
reduce by $F \rightarrow id$
reduce by $T \rightarrow F$
shift
shift
reduce by $F \rightarrow id$
reduce by $T \rightarrow T * F$
reduce by $E \rightarrow E + T$
accept

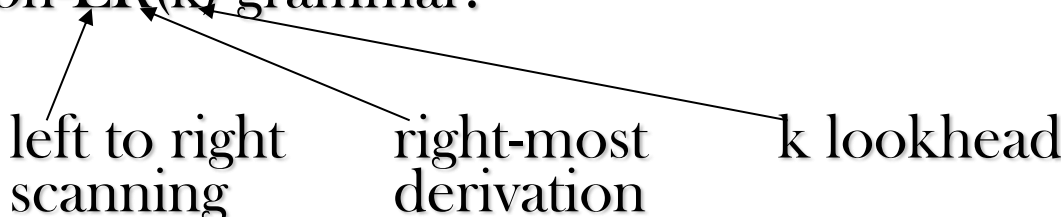
Parse Tree



Conflicts During Shift-Reduce Parsing

- There are context-free grammars for which shift-reduce parsers cannot be used.
- Stack contents and the next input symbol may not decide action:
 - **shift/reduce conflict**: Whether make a shift operation or a reduction.
 - **reduce/reduce conflict**: The parser cannot decide which of several reductions to make.
- If a shift-reduce parser cannot be used for a grammar, that grammar is called as non-LR(k) grammar.

left to right scanning right-most derivation k lookahead



- An ambiguous grammar can never be a LR grammar.

Shift-Reduce Parsers

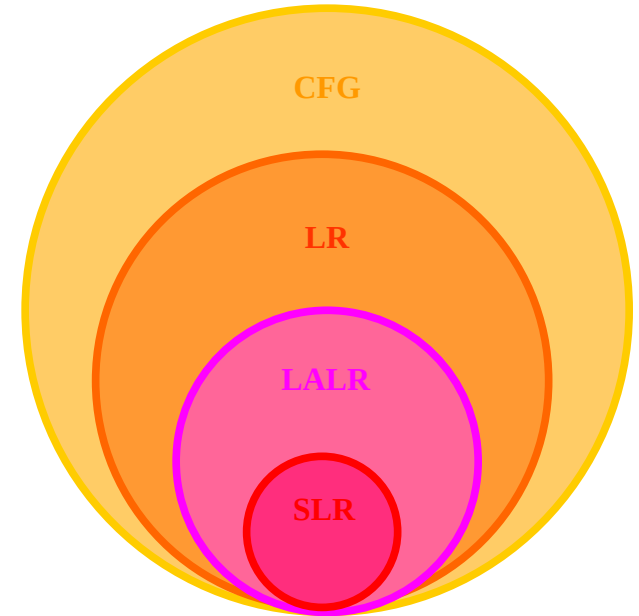
There are two main categories of shift-reduce parsers

1. Operator-Precedence Parser

- simple, but only a small class of grammars.

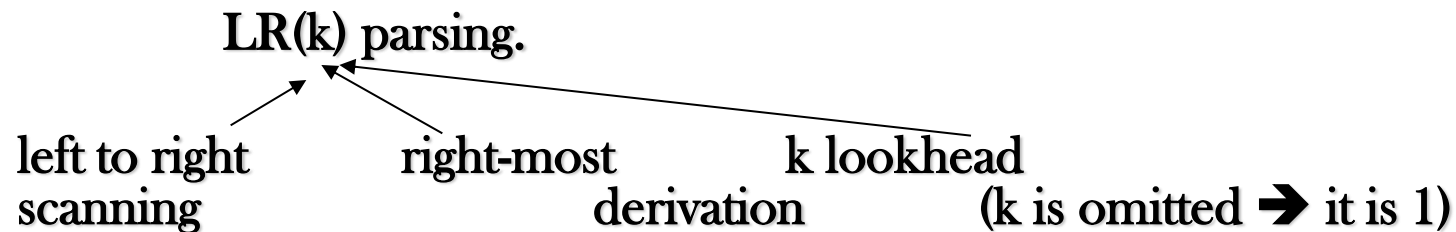
2. LR-Parsers

- covers wide range of grammars.
 - **SLR** - simple LR parser
 - **LR** - most general LR parser
 - **LALR** - intermediate LR parser (lookahead LR parser)
- SLR, LR and LALR work same, only their parsing tables are different.



LR Parsers

The most powerful shift-reduce parsing (yet efficient) is:



LR parsing is attractive because:

- LR parsing is most general non-backtracking shift-reduce parsing, yet it is still efficient.
- The class of grammars that can be parsed using LR methods is a proper superset of the class of grammars that can be parsed with predictive parsers.

LL(1)-Grammars $\subset$ LR(1)-Grammars

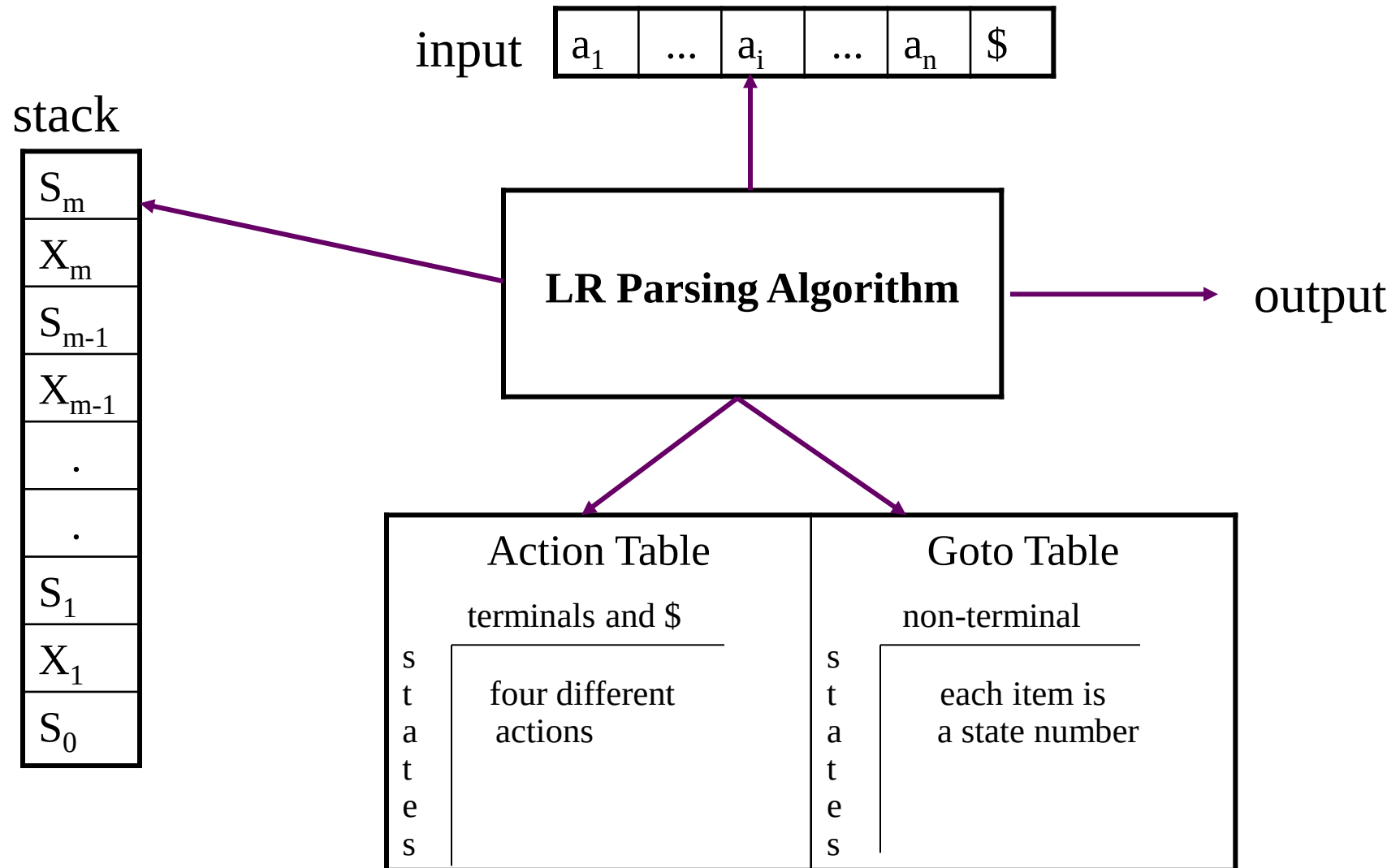
- An LR-parser can detect a syntactic error as soon as it is possible to do so a left-to-right scan of the input.

LR Parsers

LR-Parsers

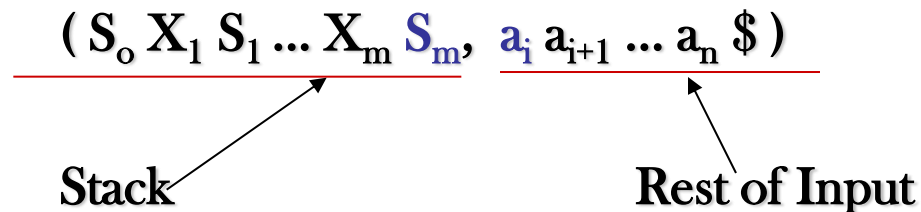
- covers wide range of grammars.
 - SLR – simple LR parser
 - LR – most general LR parser
 - LALR – intermediate LR parser (look-head LR parser)
-
- SLR, LR and LALR work same (they used the same algorithm), only their parsing tables are different.

LR Parsing Algorithm



A Configuration of LR Parsing Algorithm

- A configuration of a LR parsing is:



- S_m and a_i decides the parser action by consulting the parsing action table. (*Initial Stack* contains just S_o)
- A configuration of a LR parsing represents the right sentential form:

$$X_1 \ \dots \ X_m \ a_i \ a_{i+1} \ \dots \ a_n \ \$$$

Actions of A LR-Parser

1. **shift s** – shifts the next input symbol and the state s onto the stack (s_n where n is a state number)

$$(S_o X_1 S_1 \dots X_m S_m, a_i a_{i+1} \dots a_n \$) \rightarrow (S_o X_1 S_1 \dots X_m S_m \mathbf{a_i s}, a_{i+1} \dots a_n \$)$$

?

2. **reduce $A \rightarrow \beta$** (or rn where n is a production number)

- pop $2|\beta|$ ($=r$) items from the stack;
- then push A and s where $s = \text{goto}[s_{m-r}, A]$

$$(S_o X_1 S_1 \dots X_m S_m, a_i a_{i+1} \dots a_n \$) \rightarrow (S_o X_1 S_1 \dots X_{m-r} \mathbf{S_{m-r} A s}, a_i \dots a_n \$)$$

- Output is the reducing production reduce $A \rightarrow \beta$

3. **Accept** – Parsing successfully completed

4. **Error** – Parser detected an error (an empty entry in the action table)

Reduce Action

- ◆ pop $2|\beta|$ ($=r$) items from the stack; let us assume that $\beta = Y_1 Y_2 \dots Y_r$
- ◆ then push A and s where $s = \text{goto}[s_{m-r}, A]$

$$\begin{aligned}
 & (S_o X_1 S_1 \dots X_{m-r} S_{m-r} Y_1 S_{m-r} \dots Y_r S_{m-r} a_i a_{i+1} \dots a_n \$) \\
 & \rightarrow (S_o X_1 S_1 \dots X_{m-r} S_{m-r} A s, a_i \dots a_n \$)
 \end{aligned}$$

handle

- ◆ In fact, $Y_1 Y_2 \dots Y_r$ is a handle.

$$X_1 \dots X_{m-r} A a_i \dots a_n \$ \Rightarrow X_1 \dots X_m Y_1 \dots Y_r a_i a_{i+1} \dots a_n \$$$

(SLR) Parsing Tables for Expression Grammar

- 1) $E \rightarrow E+T$
- 2) $E \rightarrow T$
- 3) $T \rightarrow T*F$
- 4) $T \rightarrow F$
- 5) $F \rightarrow (E)$
- 6) $F \rightarrow id$

Action Table

Goto Table

state	id	+	*	(	)	\$		E	T	F
0	s5			s4				1	2	3
1		s6				acc				
2		r2	s7		r2	r2				
3		r4	r4		r4	r4				
4	s5			s4				8	2	3
5		r6	r6		r6	r6				
6	s5			s4					9	3
7	s5			s4						10
8		s6			s11					
9		r1	s7		r1	r1				
10		r3	r3		r3	r3				
11		r5	r5		r5	r5				

Actions of A (S)LR-Parser – Example

<u>stack</u>	<u>input</u>	<u>action</u>	<u>output</u>
0	id*id+id\$	shift 5	
0id5	*id+id\$	reduce by $F \rightarrow id$	$F \rightarrow id$
0F3	*id+id\$	reduce by $T \rightarrow F$	$T \rightarrow F$
0T2	*id+id\$	shift 7	
0T2*7	id+id\$	shift 5	
0T2*7id5	+id\$	reduce by $F \rightarrow id$	$F \rightarrow id$
0T2*7F10	+id\$	reduce by $T \rightarrow T * F$	$T \rightarrow T * F$
0T2	+id\$	reduce by $E \rightarrow T$	$E \rightarrow T$
0E1	+id\$	shift 6	
0E1+6	id\$	shift 5	
0E1+6id5	\$	reduce by $F \rightarrow id$	$F \rightarrow id$
0E1+6F3	\$	reduce by $T \rightarrow F$	$T \rightarrow F$
0E1+6T9	\$	reduce by $E \rightarrow E + T$	$E \rightarrow E + T$
0E1	\$	accept	

Constructing SLR Parsing Tables – LR(0)



Item

- ◆ An **LR(0) item** of a grammar G is a production of G a dot at the some position of the right side.

Ex: $A \rightarrow aBb$

Possible LR(0) Items:
(four different possibility)

$A \rightarrow \cdot aBb$

$A \rightarrow a \cdot Bb$

$A \rightarrow aB \cdot b$

$A \rightarrow aBb \cdot$

- ◆ Sets of LR(0) items will be the states of action and goto table of the SLR parser.
- ◆ A collection of sets of LR(0) items (**the canonical LR(0) collection**) is the basis for constructing SLR parsers.
- ◆ *Augmented Grammar*:
 G' is G with a new production rule $S' \rightarrow S$ where S' is the new starting symbol.

Closure Operation

If I is a set of LR(0) items for a grammar G , then *closure(I)* is the set of LR(0) items constructed from I by the two rules:

1. Initially, every LR(0) item in I is added to $\text{closure}(I)$.
2. If $A \rightarrow \alpha \bullet B\beta$ is in $\text{closure}(I)$ and $B \rightarrow \gamma$ is a production rule of G ; then $B \rightarrow \bullet \gamma$ will be in the $\text{closure}(I)$.

We will apply this rule until no more new LR(0) items can be added to $\text{closure}(I)$.

Closure Operation – Example

$E' \rightarrow E$

$E \rightarrow E+T$

$E \rightarrow T$

$T \rightarrow T^*F$

$T \rightarrow F$

$F \rightarrow (E)$

$F \rightarrow id$

$I_0: \text{closure}(\{E' \rightarrow \bullet E\}) =$

$\{ E' \rightarrow \bullet E$

$\leftarrow$ kernel items

$E \rightarrow \bullet E+T$

$E \rightarrow \bullet T$

$T \rightarrow \bullet T^*F$

$T \rightarrow \bullet F$

$F \rightarrow \bullet (E)$

$F \rightarrow \bullet id \}$

Goto Operation

If I is a set of LR(0) items and X is a grammar symbol (terminal or non-terminal), then $\text{goto}(I, X)$ is defined as follows:

- If $A \rightarrow \alpha.X\beta$ in I then
every item in $\text{closure}(\{A \rightarrow \alpha X.\beta\})$ will be in $\text{goto}(I, X)$.

Example:

$$I = \{ \begin{array}{l} E' \rightarrow .E, \quad E \rightarrow .E+T, \quad E \rightarrow .T, \\ T \rightarrow .T^*F, \quad T \rightarrow .F, \\ F \rightarrow .(E), \quad F \rightarrow .id \end{array} \}$$

$$\text{goto}(I, E) = \{ E' \rightarrow E., \quad E \rightarrow E.+T \}$$

$$\text{goto}(I, T) = \{ E \rightarrow T., \quad T \rightarrow T.^*F \}$$

$$\text{goto}(I, F) = \{ T \rightarrow F. \}$$

$$\text{goto}(I, () = \{ \begin{array}{l} F \rightarrow (.E), \quad E \rightarrow .E+T, \quad E \rightarrow .T, \quad T \rightarrow .T^*F, \quad T \rightarrow .F, \\ F \rightarrow .(E), \quad F \rightarrow .id \end{array} \}$$

$$\text{goto}(I, id) = \{ F \rightarrow id. \}$$

Construction of The Canonical LR(0) Collection

◆ To create the SLR parsing tables for a grammar G , we will create the canonical LR(0) collection of the grammar G .

◆ *Algorithm:*

C is { closure($\{S' \rightarrow .S\}$) }

repeat the followings until no more set of LR(0) items can be added to C .

for each I in C and each grammar symbol X

if goto(I, X) is not empty and not in C

add goto(I, X) to C

New item, corresponding
to new state in DFA

◆ goto function is a DFA on the sets in C .



Constructing SLR Parsing Table

(of an augmented grammar G')

1. Construct the canonical collection of sets of LR(0) items for G' . $C \leftarrow \{I_0, \dots, I_n\}$
2. Create the parsing **action table** as follows:
 - If a is a terminal, $A \rightarrow \alpha.a\beta$ in I_i and $\text{goto}(I_i, a) = I_j$ then $\text{action}[i, a]$ is *shift j*.
 - If $A \rightarrow \alpha.$ is in I_i , then $\text{action}[i, a]$ is *reduce $A \rightarrow \alpha$* for all a in $\text{FOLLOW}(A)$ where $A \neq S'$.
 - If $S' \rightarrow S.$ is in I_i , then $\text{action}[i, \$]$ is *accept*.
 - If any conflicting actions generated by these rules, the grammar is not SLR(1).
3. Create the parsing **goto table** :
 - for all non-terminals A , if $\text{goto}(I_i, A) = I_j$ then $\text{goto}[i, A] = j$
4. All entries not defined by (2) and (3) are errors.
5. Initial state of the parser contains $S' \rightarrow .S$

Parsing Tables of Expression Grammar

Action Table

Goto Table

- 1) $E \rightarrow E + T$
- 2) $E \rightarrow T$
- 3) $T \rightarrow T * F$
- 4) $T \rightarrow F$
- 5) $F \rightarrow (E)$
- 6) $F \rightarrow id$

state	id	+	*	(	)	\$		E	T	F
0	s5			s4				1	2	3
1		s6				acc				
2		r2	s7		r2	r2				
3		r4	r4		r4	r4				
4	s5			s4				8	2	3
5		r6	r6		r6	r6				
6	s5			s4					9	3
7	s5			s4						10
8		s6			s11					
9		r1	s7		r1	r1				
10		r3	r3		r3	r3				
11		r5	r5		r5	r5				

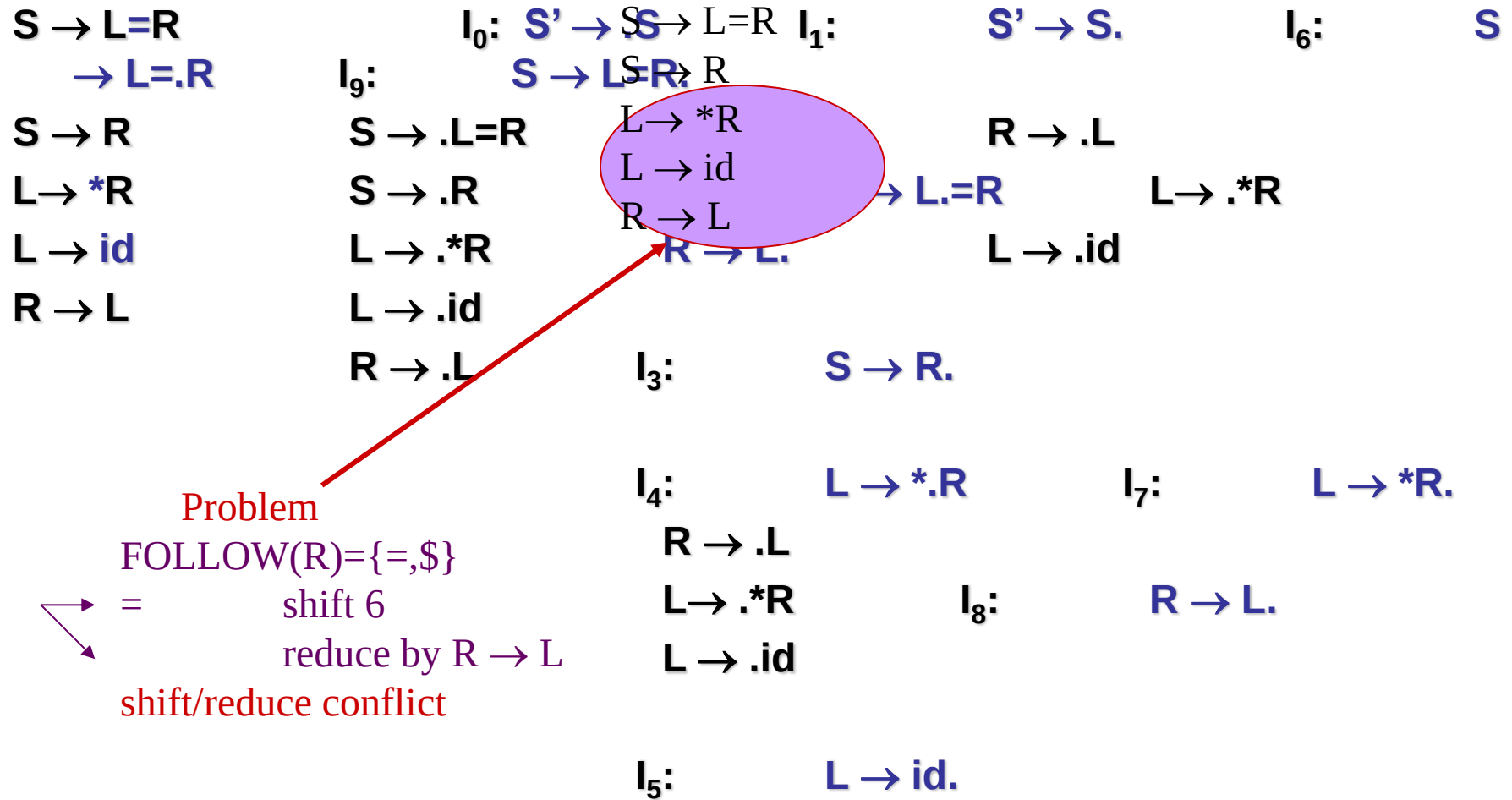
SLR(1) Grammar

- ◆ An LR parser using SLR(1) parsing tables for a grammar G is called as the SLR(1) parser for G .
- ◆ If a grammar G has an SLR(1) parsing table, it is called SLR(1) grammar (or SLR grammar in short).
- ◆ Every SLR grammar is unambiguous, but every unambiguous grammar is not a SLR grammar.

shift/reduce and reduce/reduce conflicts

- ◆ If a state does not know whether it will make a shift operation or reduction for a terminal, we say that there is a **shift/reduce conflict**.
- ◆ If a state does not know whether it will make a reduction operation using the production rule i or j for a terminal, we say that there is a **reduce/reduce conflict**.
- ◆ If the SLR parsing table of a grammar G has a conflict, we say that that grammar is not SLR grammar.

Conflict Example



Exercise-Review LL(1)

产生式

1. $S \rightarrow L=R$

2. $S \rightarrow R$

3. $L \rightarrow *R$

4. $L \rightarrow id$

5. $R \rightarrow L$

	FIRST	FOLLOW
S	*, id	\$
L	*, id	\$, =
R	*, id	\$, =

LL(1)分析表

	*	id	\$
S	1, 2	1, 2	
L	3	4	
R	5	5	

因为表项有冲突，所以该文法不是LL(1)文法。

Conflict Example2

Cf. P.273 4.37 a)

$S \rightarrow AaAb$

$S \rightarrow BbBa$

$A \rightarrow \varepsilon$

$B \rightarrow \varepsilon$

$I_0: \quad S' \rightarrow .S$

$S \rightarrow .AaAb$

$S \rightarrow .BbBa$

$A \rightarrow .$

$B \rightarrow .$

$S \rightarrow AaAb$

$S \rightarrow BbBa$

$A \rightarrow \varepsilon$

$B \rightarrow \varepsilon$

Problem

$FOLLOW(A) = \{a, b\}$

$FOLLOW(B) = \{a, b\}$

$a \rightarrow$ reduce by $A \rightarrow \varepsilon$

$\searrow$ reduce by $B \rightarrow \varepsilon$

reduce/reduce conflict

$b \rightarrow$ reduce by $A \rightarrow \varepsilon$

$\searrow$ reduce by $B \rightarrow \varepsilon$

reduce/reduce conflict

Exercise- review LL(1)

产生式

1. $S \rightarrow AaAb$

2. $S \rightarrow BbBa$

3. $A \rightarrow \epsilon$

4. $B \rightarrow \epsilon$

	FIRST	FOLLOW
S	a, b	\$
A	ϵ	a, b
B	ϵ	a, b

LL(1)分析表

	a	b	\$
S	1	2	
A	3	3	
B	4	4	

因为表项没有冲突，所以该文法是LL(1)文法。